Digital Image Processing — Programs 1 to 6

Corrected Python code. Replace **"image.jpg"** with your image filename. Required libraries: OpenCV, NumPy, and Matplotlib.

PROGRAM 1: Read Color and Grayscale Images and Convert Color to Grayscale

```
import cv2
import matplotlib.pyplot as plt

color_img = cv2.imread("image.jpg")
if color_img is None:
    raise FileNotFoundError("Image not found")

rgb_img = cv2.cvtColor(color_img, cv2.COLOR_BGR2RGB)
gray_img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
converted_gray = cv2.cvtColor(color_img, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(rgb_img)
plt.title("Color Image (RGB)")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(gray_img, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(converted_gray, cmap="gray")
plt.title("Converted Grayscale Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 2: Simulate Image Sampling and Quantization

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
if image is None:
    raise FileNotFoundError("Image not found")

print("Original image shape:", image.shape)
sampled_image = cv2.resize(image, (128, 128))

gray_levels = 16
step = 256 / gray_levels
quantized_image = (np.floor(sampled_image / step) * step).astype(np.uint8)

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(sampled_image, cmap="gray")
plt.title("Sampled Image (128 x 128)")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(quantized_image, cmap="gray")
plt.title("Quantized Image (16 Levels)")
plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 3: Demonstrate Spatial and Intensity Resolution

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Image not found")

img_256 = cv2.resize(img, (256, 256))
img_128 = cv2.resize(img, (128, 128))
img_64 = cv2.resize(img, (64, 64))

step = 256 / 64
img_64_levels = (np.floor(img_256 / step) * step).astype(np.uint8)
step = 256 / 16
img_16_levels = (np.floor(img_256 / step) * step).astype(np.uint8)
step = 256 / 4
img_4_levels = (np.floor(img_256 / step) * step).astype(np.uint8)

plt.figure(figsize=(12, 8))
plt.subplot(2, 3, 1)
plt.imshow(img_256, cmap="gray")
plt.title("256 x 256")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(img_128, cmap="gray")
plt.title("128 x 128")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(img_64, cmap="gray")
plt.title("64 x 64")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(img_64_levels, cmap="gray")
plt.title("64 Gray Levels")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(img_16_levels, cmap="gray")
plt.title("16 Gray Levels")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(img_4_levels, cmap="gray")
plt.title("4 Gray Levels")
plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 4: Compute and Visualize Adjacency and Connectivity in a Binary Image

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Image not found")

_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
num_labels, labels = cv2.connectedComponents(binary, connectivity=8)
num_components = num_labels - 1

print("Number of connected components:", num_components)

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(binary, cmap="gray")
plt.title("Binary Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(labels, cmap="nipy_spectral")
plt.title("Connected Components")
plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 5: Separate RGB Components and Convert to Other Color Models

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("image.jpg")
if image is None:
    raise FileNotFoundError("Image not found")

rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
R, G, B = rgb[:, :, 0], rgb[:, :, 1], rgb[:, :, 2]

# CMY components
C = 255 - R
M = 255 - G
Y = 255 - B

# HSV components
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
H, S, V = hsv[:, :, 0], hsv[:, :, 1], hsv[:, :, 2]

plt.figure(figsize=(12, 10))
plt.subplot(3, 4, 1)
plt.imshow(rgb)
plt.title("Original RGB")
plt.axis("off")

components = [
    (R, "Red"), (G, "Green"), (B, "Blue"),
    (C, "Cyan"), (M, "Magenta"), (Y, "Yellow"),
    (H, "Hue"), (S, "Saturation"), (V, "Value")
]
for i, (component, title) in enumerate(components, start=2):
    plt.subplot(3, 4, i)
    plt.imshow(component, cmap="gray")
    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 6: Calculate Hue, Saturation and Intensity (HSI)

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("image.jpg")
if image is None:
    raise FileNotFoundError("Image not found")

rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB).astype(np.float32) / 255.0
R, G, B = rgb[:, :, 0], rgb[:, :, 1], rgb[:, :, 2]

# Intensity
I = (R + G + B) / 3.0

# Saturation
minimum = np.minimum(np.minimum(R, G), B)
S = 1 - (3 * minimum / (R + G + B + 1e-6))

# Hue
num = 0.5 * ((R - G) + (R - B))
den = np.sqrt((R - G) ** 2 + (R - B) * (G - B))
ratio = np.clip(num / (den + 1e-6), -1, 1)
theta = np.arccos(ratio)
H = np.where(B <= G, theta, 2 * np.pi - theta)
H = H / (2 * np.pi)

plt.figure(figsize=(12, 5))
plt.subplot(1, 4, 1)
plt.imshow(rgb)
plt.title("Original RGB Image")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(H, cmap="hsv")
plt.title("Hue")
plt.axis("off")

plt.subplot(1, 4, 3)
plt.imshow(S, cmap="gray", vmin=0, vmax=1)
plt.title("Saturation")
plt.axis("off")

plt.subplot(1, 4, 4)
plt.imshow(I, cmap="gray", vmin=0, vmax=1)
plt.title("Intensity")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Digital Image Processing — Programs 7 to 14

Corrected Python code on a white background. Replace **"image.jpg"** or **"hill.jpg"** with the actual image filename. Required libraries: OpenCV, NumPy, and Matplotlib.

PROGRAM 7: Plot Color Histograms for Each Channel

```
import cv2
import matplotlib.pyplot as plt

# Read color image
image = cv2.imread("image.jpg")

if image is None:
    raise FileNotFoundError("Image not found")

# Convert BGR to RGB
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Separate RGB channels
r, g, b = cv2.split(image)

# Red channel histogram
plt.hist(r.ravel(), bins=256, range=(0, 256), color="red")
plt.title("Red Channel Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.show()

# Green channel histogram
plt.hist(g.ravel(), bins=256, range=(0, 256), color="green")
plt.title("Green Channel Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.show()

# Blue channel histogram
plt.hist(b.ravel(), bins=256, range=(0, 256), color="blue")
plt.title("Blue Channel Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.show()
```

PROGRAM 8: Image Negative Transformation

```
import cv2
import matplotlib.pyplot as plt

# Read image
image = cv2.imread("image.jpg", 0)

if image is None:
    raise FileNotFoundError("Image not found")

# Calculate negative image
negative = 255 - image

# Display original image
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.show()

# Display negative image
plt.imshow(negative, cmap="gray")
plt.title("Negative Image")
plt.axis("off")
plt.show()
```

PROGRAM 9: Apply Log Transformation for Contrast Enhancement

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read grayscale image
image = cv2.imread("image.jpg", 0)

if image is None:
    raise FileNotFoundError("Image not found")

# Convert image to float
image = image.astype(np.float32)

# Calculate constant
c = 255 / np.log1p(np.max(image))

# Apply log transformation
log_image = c * np.log1p(image)

# Convert result to uint8
log_image = np.uint8(np.clip(log_image, 0, 255))

# Display original image
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.show()

# Display log-transformed image
plt.imshow(log_image, cmap="gray")
plt.title("Log Transformed Image")
plt.axis("off")
plt.show()
```

PROGRAM 10: Power-Law (Gamma) Transformation with Different Gamma Values

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read grayscale image
image = cv2.imread("image.jpg", 0)

if image is None:
    raise FileNotFoundError("Image not found")

# Normalize image
normalized = image / 255.0

# Different gamma values
gamma_values = [0.5, 1.0, 2.0]

plt.figure(figsize=(12, 4))

for i, gamma in enumerate(gamma_values):
    # Apply gamma transformation
    gamma_image = np.power(normalized, gamma)
    gamma_image = np.uint8(gamma_image * 255)

    plt.subplot(1, 3, i + 1)
    plt.imshow(gamma_image, cmap="gray")
    plt.title("Gamma = " + str(gamma))
    plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 11: Contrast Stretching and Thresholding

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read grayscale image
image = cv2.imread("hill.jpg", 0)

if image is None:
    raise FileNotFoundError("Image not found")

# Contrast stretching
r_min = np.min(image)
r_max = np.max(image)

if r_max == r_min:
    stretched = image.copy()
else:
    stretched = (
        (image.astype(np.float32) - r_min)
        * 255 / (r_max - r_min)
    ).astype(np.uint8)

# Thresholding
threshold = 128
_, thresholded = cv2.threshold(
    image, threshold, 255, cv2.THRESH_BINARY
)

# Display original image
plt.subplot(1, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

# Display contrast-stretched image
plt.subplot(1, 3, 2)
plt.imshow(stretched, cmap="gray")
plt.title("Contrast Stretched")
plt.axis("off")

# Display thresholded image
plt.subplot(1, 3, 3)
plt.imshow(thresholded, cmap="gray")
plt.title("Thresholded Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 12: Piecewise Thresholding (Single Threshold Used in the Notebook)

```
import cv2
import matplotlib.pyplot as plt

# Read grayscale image
image = cv2.imread("hill.jpg", 0)

if image is None:
    raise FileNotFoundError("Image not found")

# Apply thresholding
threshold = 128

_, thresholded = cv2.threshold(
    image, threshold, 255, cv2.THRESH_BINARY
)

# Display original image
plt.subplot(1, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

# Display thresholded image
plt.subplot(1, 2, 2)
plt.imshow(thresholded, cmap="gray")
plt.title("Thresholded Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 13: Histogram Equalization

```
import cv2
import matplotlib.pyplot as plt

# Read grayscale image
image = cv2.imread("hill.jpg", 0)

if image is None:
    raise FileNotFoundError("Image not found")

# Apply histogram equalization
equalized = cv2.equalizeHist(image)

# Display original image
plt.subplot(2, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

# Display equalized image
plt.subplot(2, 2, 2)
plt.imshow(equalized, cmap="gray")
plt.title("Equalized Image")
plt.axis("off")

# Original histogram
plt.subplot(2, 2, 3)
plt.hist(image.ravel(), bins=256, range=(0, 256))
plt.title("Original Histogram")
plt.xlabel("Intensity")
plt.ylabel("Frequency")

# Equalized histogram
plt.subplot(2, 2, 4)
plt.hist(equalized.ravel(), bins=256, range=(0, 256))
plt.title("Equalized Histogram")
plt.xlabel("Intensity")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

PROGRAM 14: Implement Histogram Stretching

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read grayscale image
image = cv2.imread("hill.jpg", 0)

if image is None:
    raise FileNotFoundError("Image not found")

# Find minimum and maximum pixel values
r_min = np.min(image)
r_max = np.max(image)

# Apply histogram stretching
if r_max == r_min:
    stretched = image.copy()
else:
    stretched = (
        (image.astype(np.float32) - r_min)
        * 255 / (r_max - r_min)
    ).astype(np.uint8)

# Display original image
plt.subplot(2, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

# Display stretched image
plt.subplot(2, 2, 2)
plt.imshow(stretched, cmap="gray")
plt.title("Stretched Image")
plt.axis("off")

# Original histogram
plt.subplot(2, 2, 3)
plt.hist(image.ravel(), bins=256, range=(0, 256))
plt.title("Original Histogram")
plt.xlabel("Intensity")
plt.ylabel("Frequency")

# Stretched histogram
plt.subplot(2, 2, 4)
plt.hist(stretched.ravel(), bins=256, range=(0, 256))
plt.title("Stretched Histogram")
plt.xlabel("Intensity")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

Digital Image Processing — Programs 14 to 16

Corrected Python code on a white background. Replace the example image filenames with the actual filenames on your computer. Required libraries: OpenCV, NumPy, and Matplotlib.

PROGRAM 14: Histogram Matching

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read source and reference images
source = cv2.imread("tiger.jpg", 0)
reference = cv2.imread("tigerblue.jpg", 0)

if source is None or reference is None:
    raise FileNotFoundError("Check the image filenames.")

# Calculate histograms
source_hist = cv2.calcHist(
    [source], [0], None, [256], [0, 256]
)

reference_hist = cv2.calcHist(
    [reference], [0], None, [256], [0, 256]
)

# Calculate cumulative distribution functions
source_cdf = source_hist.cumsum()
reference_cdf = reference_hist.cumsum()

# Normalize the CDFs
source_cdf = source_cdf / source_cdf[-1]
reference_cdf = reference_cdf / reference_cdf[-1]

# Create intensity mapping
mapping = np.zeros(256, dtype=np.uint8)

for i in range(256):
    mapping[i] = np.argmin(
        np.abs(reference_cdf - source_cdf[i])
    )

# Match the source image to the reference image
matched_image = mapping[source]

# Display images and histograms
plt.figure(figsize=(12, 8))

plt.subplot(3, 3, 1)
plt.imshow(source, cmap="gray")
plt.title("Source Image")
plt.axis("off")

plt.subplot(3, 3, 2)
plt.imshow(reference, cmap="gray")
plt.title("Reference Image")
plt.axis("off")

plt.subplot(3, 3, 3)
plt.imshow(matched_image, cmap="gray")
plt.title("Matched Image")
plt.axis("off")

plt.subplot(3, 3, 4)
plt.hist(source.ravel(), bins=256, range=(0, 256))
plt.title("Source Histogram")
plt.subplot(3, 3, 5)
```

```
plt.hist(reference.ravel(), bins=256, range=(0, 256))
plt.title("Reference Histogram")

plt.subplot(3, 3, 6)
plt.hist(matched_image.ravel(), bins=256, range=(0, 256))
plt.title("Matched Histogram")

plt.tight_layout()
plt.show()
```

PROGRAM 15: Spatial Correlation and Convolution Using a Custom Kernel

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read grayscale image
image = cv2.imread("bird.jpg", cv2.IMREAD_GRAYSCALE)

if image is None:
    raise FileNotFoundError("Image not found.")

# Define a custom 3x3 kernel
kernel = np.array([
    [1, 1, 1],
    [1, 1, 1],
    [1, 1, 1]
], dtype=np.float32)

# Normalize the kernel
kernel = kernel / 9

# Perform spatial correlation
correlation = cv2.filter2D(image, -1, kernel)

# Flip the kernel for convolution
flipped_kernel = np.flip(kernel)

# Perform spatial convolution
convolution = cv2.filter2D(
    image, -1, flipped_kernel
)

# Display results
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(correlation, cmap="gray")
plt.title("Spatial Correlation")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(convolution, cmap="gray")
plt.title("Spatial Convolution")
plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 16: Apply Mean and Gaussian Smoothing Filters

```
import cv2
import matplotlib.pyplot as plt

# Read the image
image = cv2.imread("bird.jpg")

if image is None:
    raise FileNotFoundError("Image not found.")

# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Apply mean filter
mean_filter = cv2.blur(image_rgb, (5, 5))

# Apply Gaussian filter
gaussian_filter = cv2.GaussianBlur(
    image_rgb, (5, 5), 0
)

# Display results
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(mean_filter)
plt.title("Mean Filter")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(gaussian_filter)
plt.title("Gaussian Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 17: Sharpening Filters Using Laplacian

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read the image
img = cv2.imread("bird.jpg", 0)

# Define Laplacian kernel
kernel = np.array([
    [0, -1, 0],
    [-1, 4, -1],
    [0, -1, 0]
])

# Apply Laplacian filter
laplacian = cv2.filter2D(img, cv2.CV_64F, kernel)

# Sharpen the image
sharpened = img + laplacian

# Convert values to range 0-255
sharpened = np.clip(sharpened, 0, 255)

# Convert to unsigned 8-bit image
sharpened = np.uint8(sharpened)

# Display original image
plt.subplot(1, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

# Display Laplacian image
plt.subplot(1, 3, 2)
plt.imshow(np.abs(laplacian), cmap="gray")
plt.title("Laplacian Image")
plt.axis("off")

# Display sharpened image
plt.subplot(1, 3, 3)
plt.imshow(sharpened, cmap="gray")
plt.title("Sharpened Image")
plt.axis("off")

plt.show()
```

PROGRAM 18: Unsharp Masking and High-Boost Filtering

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("bird.jpg", 0)

# Convert image to float
img = img.astype(np.float64)

# Create blurred image using Gaussian filter
blurred = cv2.GaussianBlur(img, (5, 5), 0)

# Create unsharp mask
mask = img - blurred

# Unsharp masking
unsharp = img + mask

# High-boost filtering
k = 2
high_boost = img + k * mask

# Keep pixel values between 0 and 255
unsharp = np.clip(unsharp, 0, 255)
high_boost = np.clip(high_boost, 0, 255)

# Convert to uint8
unsharp = np.uint8(unsharp)
high_boost = np.uint8(high_boost)

# Display images
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(blurred, cmap="gray")
plt.title("Blurred Image")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(unsharp, cmap="gray")
plt.title("Unsharp Masking")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(high_boost, cmap="gray")
plt.title("High-Boost Filtering")
plt.axis("off")

plt.show()
```

PROGRAM 19: Sobel and Prewitt Edge Detection

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("bird.jpg", 0)

# ---------------- SOBEL OPERATOR ----------------

# Sobel in X direction
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)

# Sobel in Y direction
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)

# Gradient magnitude
sobel = np.sqrt(sobel_x**2 + sobel_y**2)

# Convert to uint8
sobel = np.uint8(np.clip(sobel, 0, 255))

# ---------------- PREWITT OPERATOR ----------------

# Prewitt kernels
prewitt_x = np.array([
    [-1, 0, 1],
    [-1, 0, 1],
    [-1, 0, 1]
])

prewitt_y = np.array([
    [-1, -1, -1],
    [ 0,  0,  0],
    [ 1,  1,  1]
])

# Apply Prewitt kernels
px = cv2.filter2D(img, cv2.CV_64F, prewitt_x)
py = cv2.filter2D(img, cv2.CV_64F, prewitt_y)

# Gradient magnitude
prewitt = np.sqrt(px**2 + py**2)

# Convert to uint8
prewitt = np.uint8(np.clip(prewitt, 0, 255))

# ---------------- DISPLAY RESULTS ----------------

plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(np.abs(sobel_x), cmap="gray")
plt.title("Sobel X")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(np.abs(sobel_y), cmap="gray")
plt.title("Sobel Y")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(sobel, cmap="gray")
plt.title("Sobel Edge")
```

```
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(prewitt, cmap="gray")
plt.title("Prewitt Edge")
plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 20: 2D Fourier Transform and Inverse DFT

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Read the image in grayscale
img = cv2.imread("bird.jpg", cv2.IMREAD_GRAYSCALE)

# Check whether image is loaded
if img is None:
    print("Image not found")
    exit()

# 2. Compute the 2-D Fourier Transform
dft = np.fft.fft2(img)

# 3. Shift the zero-frequency component to the center
dft_shift = np.fft.fftshift(dft)

# 4. Calculate the magnitude spectrum
magnitude_spectrum = 20 * np.log(np.abs(dft_shift) + 1)

# 5. Compute the Inverse DFT
idft_shift = np.fft.ifftshift(dft_shift)
reconstructed = np.fft.ifft2(idft_shift)

# 6. Convert the complex result to real image
reconstructed = np.abs(reconstructed)

# 7. Display the results
plt.figure(figsize=(12, 8))

plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(magnitude_spectrum, cmap='gray')
plt.title("Fourier Spectrum")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(reconstructed, cmap='gray')
plt.title("Reconstructed Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 21: Low-Pass and High-Pass Filters in Frequency Domain

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("bird.jpg", cv2.IMREAD_GRAYSCALE)

if img is None:
    print("Image not found")
    exit()

# Get image dimensions
M, N = img.shape

# Step 1: Compute 2-D DFT
dft = np.fft.fft2(img)

# Step 2: Shift zero frequency to center
dft_shift = np.fft.fftshift(dft)

# Create coordinate grid
u = np.arange(N)
v = np.arange(M)
U, V = np.meshgrid(u, v)

# Center coordinates
D = np.sqrt((U - N/2)**2 + (V - M/2)**2)

# Cutoff frequency
D0 = 50

# Step 3: Create Low-Pass Filter
LPF = np.zeros((M, N))
LPF[D <= D0] = 1

# Step 4: Create High-Pass Filter
HPF = np.ones((M, N))
HPF[D <= D0] = 0

# Step 5: Apply filters
dft_LPF = dft_shift * LPF
dft_HPF = dft_shift * HPF

# Step 6: Shift back
dft_LPF_shift = np.fft.ifftshift(dft_LPF)
dft_HPF_shift = np.fft.ifftshift(dft_HPF)

# Step 7: Inverse DFT
img_LPF = np.fft.ifft2(dft_LPF_shift)
img_HPF = np.fft.ifft2(dft_HPF_shift)

# Convert to magnitude
img_LPF = np.abs(img_LPF)
img_HPF = np.abs(img_HPF)

# Display results
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(LPF, cmap='gray')
plt.title("Low-Pass Filter")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(img_LPF, cmap='gray')
plt.title("Low-Pass Filtered Image")
plt.axis("off")
```

```
plt.subplot(2, 3, 4)
plt.imshow(np.log(1 + np.abs(dft_shift)), cmap='gray')
plt.title("Fourier Spectrum")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(HPF, cmap='gray')
plt.title("High-Pass Filter")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(img_HPF, cmap='gray')
plt.title("High-Pass Filtered Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 22(A): DFT Property — Translation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("bird.jpg", cv2.IMREAD_GRAYSCALE)

# Translate image
M, N = img.shape

tx = 50
ty = 30

translation_matrix = np.float32([
    [1, 0, tx],
    [0, 1, ty]
])

translated = cv2.warpAffine(
    img,
    translation_matrix,
    (N, M)
)

# DFT of original image
dft_original = np.fft.fft2(img)
dft_original_shift = np.fft.fftshift(dft_original)

# DFT of translated image
dft_translated = np.fft.fft2(translated)
dft_translated_shift = np.fft.fftshift(dft_translated)

# Magnitude spectra
spectrum_original = np.log(
    1 + np.abs(dft_original_shift)
)

spectrum_translated = np.log(
    1 + np.abs(dft_translated_shift)
)

# Display
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(translated, cmap="gray")
plt.title("Translated Image")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(spectrum_original, cmap="gray")
plt.title("DFT of Original Image")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(spectrum_translated, cmap="gray")
plt.title("DFT of Translated Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 22(B): DFT Property — Scaling

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("bird.jpg", cv2.IMREAD_GRAYSCALE)

# Original dimensions
M, N = img.shape

# Scale image
scaled = cv2.resize(
    img,
    None,
    fx=1.5,
    fy=1.5,
    interpolation=cv2.INTER_LINEAR
)

# DFT of original image
dft_original = np.fft.fft2(img)
dft_original_shift = np.fft.fftshift(dft_original)

# DFT of scaled image
dft_scaled = np.fft.fft2(scaled)
dft_scaled_shift = np.fft.fftshift(dft_scaled)

# Magnitude spectra
spectrum_original = np.log(
    1 + np.abs(dft_original_shift)
)

spectrum_scaled = np.log(
    1 + np.abs(dft_scaled_shift)
)

# Display
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(scaled, cmap="gray")
plt.title("Scaled Image")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(spectrum_original, cmap="gray")
plt.title("DFT of Original")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(spectrum_scaled, cmap="gray")
plt.title("DFT of Scaled Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 23: Morphological Operations — Dilation, Erosion, Opening and Closing

```
import cv2
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("bird.jpg", cv2.IMREAD_GRAYSCALE)

# Convert to binary image
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

# Create different structuring elements
rect_kernel = cv2.getStructuringElement(
    cv2.MORPH_RECT, (3, 3)
)

ellipse_kernel = cv2.getStructuringElement(
    cv2.MORPH_ELLIPSE, (5, 5)
)

cross_kernel = cv2.getStructuringElement(
    cv2.MORPH_CROSS, (5, 5)
)

# Dilation
dilation_rect = cv2.dilate(binary, rect_kernel)
dilation_ellipse = cv2.dilate(binary, ellipse_kernel)
dilation_cross = cv2.dilate(binary, cross_kernel)

# Erosion
erosion_rect = cv2.erode(binary, rect_kernel)
erosion_ellipse = cv2.erode(binary, ellipse_kernel)
erosion_cross = cv2.erode(binary, cross_kernel)

# Opening
opening = cv2.morphologyEx(
    binary,
    cv2.MORPH_OPEN,
    rect_kernel
)

# Closing
closing = cv2.morphologyEx(
    binary,
    cv2.MORPH_CLOSE,
    rect_kernel
)

# Display results
plt.figure(figsize=(12, 8))

plt.subplot(3, 3, 1)
plt.imshow(binary, cmap="gray")
plt.title("Original Binary")
plt.axis("off")

plt.subplot(3, 3, 2)
plt.imshow(dilation_rect, cmap="gray")
plt.title("Dilation - Rect")
plt.axis("off")

plt.subplot(3, 3, 3)
plt.imshow(erosion_rect, cmap="gray")
plt.title("Erosion - Rect")
plt.axis("off")

plt.subplot(3, 3, 4)
plt.imshow(dilation_ellipse, cmap="gray")
plt.title("Dilation - Ellipse")
plt.axis("off")

plt.subplot(3, 3, 5)
plt.imshow(erosion_ellipse, cmap="gray")
plt.title("Erosion - Ellipse")
plt.axis("off")

plt.subplot(3, 3, 6)
plt.imshow(dilation_cross, cmap="gray")
```

```
plt.title("Dilation - Cross")
plt.axis("off")

plt.subplot(3, 3, 7)
plt.imshow(erosion_cross, cmap="gray")
plt.title("Erosion - Cross")
plt.axis("off")

plt.subplot(3, 3, 8)
plt.imshow(opening, cmap="gray")
plt.title("Opening")
plt.axis("off")

plt.subplot(3, 3, 9)
plt.imshow(closing, cmap="gray")
plt.title("Closing")
plt.axis("off")

plt.tight_layout()
plt.show()
```

PROGRAM 24: Hit-or-Miss Transformation for Pattern Detection

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Create a binary image
img = np.array([
    [0, 0, 0, 0, 0],
    [0, 1, 1, 0, 0],
    [0, 1, 0, 0, 0],
    [0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0]
], dtype=np.uint8)

# Structuring element
kernel = np.array([
    [1, 1],
    [1, -1]
], dtype=np.int8)

# Hit-or-miss transformation
result = cv2.morphologyEx(
    img,
    cv2.MORPH_HITMISS,
    kernel
)

print("Input Image:")
print(img)

print("\nResult:")
print(result)

# Display
plt.subplot(1, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Binary Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(result, cmap="gray")
plt.title("Hit-or-Miss Result")
plt.axis("off")

plt.show()
```

PROGRAM 25: Noise Types and Mean Filtering

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("bird.jpg", cv2.IMREAD_GRAYSCALE)

if img is None:
    print("Image not found")
    exit()

# Convert image to float
img_float = img.astype(np.float32) / 255.0

# ------------------------------------------------
# 1. Gaussian Noise
# ------------------------------------------------
mean = 0
sigma = 0.1

gaussian_noise = np.random.normal(
    mean, sigma, img_float.shape
)

gaussian_img = img_float + gaussian_noise
gaussian_img = np.clip(gaussian_img, 0, 1)
gaussian_img = (gaussian_img * 255).astype(np.uint8)

# ------------------------------------------------
# 2. Salt-and-Pepper Noise
# ------------------------------------------------
sp_img = img.copy()
probability = 0.05
random_matrix = np.random.rand(*img.shape)

# Pepper
sp_img[random_matrix < probability / 2] = 0

# Salt
sp_img[
    (random_matrix >= probability / 2) &
    (random_matrix < probability)
] = 255

# ------------------------------------------------
# 3. Rayleigh Noise
# ------------------------------------------------
rayleigh_noise = np.random.rayleigh(
    scale=0.1,
    size=img_float.shape
)

rayleigh_img = img_float + rayleigh_noise
rayleigh_img = np.clip(rayleigh_img, 0, 1)
rayleigh_img = (rayleigh_img * 255).astype(np.uint8)

# ------------------------------------------------
# 4. Gamma Noise
# ------------------------------------------------
gamma_noise = np.random.gamma(
    shape=2.0,
    scale=0.1,
    size=img_float.shape
)

gamma_img = img_float + gamma_noise
gamma_img = np.clip(gamma_img, 0, 1)
gamma_img = (gamma_img * 255).astype(np.uint8)

# ------------------------------------------------
# 5. Exponential Noise
# ------------------------------------------------
exponential_noise = np.random.exponential(
    scale=0.1,
    size=img_float.shape
)
```

```
exponential_img = img_float + exponential_noise
exponential_img = np.clip(exponential_img, 0, 1)
exponential_img = (
    exponential_img * 255
).astype(np.uint8)

# ------------------------------------------------
# Mean filtering
# ------------------------------------------------
gaussian_filtered = cv2.blur(
    gaussian_img, (3, 3)
)

sp_filtered = cv2.blur(
    sp_img, (3, 3)
)

rayleigh_filtered = cv2.blur(
    rayleigh_img, (3, 3)
)

gamma_filtered = cv2.blur(
    gamma_img, (3, 3)
)

exponential_filtered = cv2.blur(
    exponential_img, (3, 3)
)

# ------------------------------------------------
# Display results
# ------------------------------------------------
plt.figure(figsize=(12, 10))

plt.subplot(5, 3, 1)
plt.imshow(gaussian_img, cmap="gray")
plt.title("Gaussian Noise")
plt.axis("off")

plt.subplot(5, 3, 2)
plt.imshow(gaussian_filtered, cmap="gray")
plt.title("Gaussian + Mean Filter")
plt.axis("off")

plt.subplot(5, 3, 4)
plt.imshow(sp_img, cmap="gray")
plt.title("Salt & Pepper")
plt.axis("off")

plt.subplot(5, 3, 5)
plt.imshow(sp_filtered, cmap="gray")
plt.title("S&P + Mean Filter")
plt.axis("off")

plt.subplot(5, 3, 7)
plt.imshow(rayleigh_img, cmap="gray")
plt.title("Rayleigh Noise")
plt.axis("off")

plt.subplot(5, 3, 8)
plt.imshow(rayleigh_filtered, cmap="gray")
plt.title("Rayleigh + Mean Filter")
plt.axis("off")

plt.subplot(5, 3, 10)
plt.imshow(gamma_img, cmap="gray")
plt.title("Gamma Noise")
plt.axis("off")

plt.subplot(5, 3, 11)
plt.imshow(gamma_filtered, cmap="gray")
plt.title("Gamma + Mean Filter")
plt.axis("off")

plt.subplot(5, 3, 13)
plt.imshow(exponential_img, cmap="gray")
plt.title("Exponential Noise")
plt.axis("off")
```

```
plt.subplot(5, 3, 14)
plt.imshow(exponential_filtered, cmap="gray")
plt.title("Exponential + Mean Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```